Walk-In Interview Prep Plan — TCS Gen AI / AI-ML / Python Developer

Interview date: 18th July 2026 | **You have:** 4 days (14th–17th, interview on 18th)

---

0. Reality Check First (Read This)

Your resume is strong in: **Python, Django, FastAPI, SQLAlchemy, ETL/Airflow, Snowflake, dbt, REST APIs, Selenium automation, GCP/AWS.**

Your resume has **zero Gen AI/LLM/RAG/MLOps experience** — you already told the recruiter "0 years" honestly, which is good. That means:

- The interviewer already knows you're a **backend/data engineer transitioning into Gen AI**, not a seasoned Gen AI engineer.
- You will NOT be expected to answer like someone with 2 years of production LLM experience.
- Your job in 4 days is to build a **credible working knowledge** of core Gen AI concepts + be able to say "I haven't built this in production, but I understand it and here's how I'd approach it, and here's a project I built to learn it."
- Your biggest asset: you already know how to build REST APIs (FastAPI), pipelines (Airflow), and handle data at scale. Gen AI apps are just APIs + data pipelines with an LLM in the loop. Frame everything through that lens.

Strategy: 60% effort on Gen AI/LLM fundamentals + 1 small hands-on RAG project, 25% on Python/DSA revision, 15% on your existing project deep-dive (they WILL grill you on Accenture/FedEx/AstraZeneca/Stelvio since that's your strongest ground).

---

1. Day-by-Day Plan

Day 1 (14th July) — Python Core + Existing Project Deep Dive

Goal: Be bulletproof on what's already on your resume. This is free marks.

- **Morning (2–3 hrs): Python fundamentals interviewers test at 3 YOE**
  - Mutable vs immutable types, `*args`/`**kwargs`, decorators, generators/`yield`, context managers (`with`), list/dict comprehensions
  - GIL, multithreading vs multiprocessing (basic understanding, why Python is slow for CPU-bound work)
  - `@staticmethod` vs `@classmethod`, `__init__` vs `__new__`

    - Exception handling patterns, custom exceptions
- **Afternoon (2–3 hrs): Re-read your own resume like an interviewer** For EACH bullet point on FedEx, AstraZeneca, and Stelvio projects, prepare to answer:
    - What exact problem were you solving?
    - What was the architecture? (Draw it if asked)
    - What would you do differently now?
    - What was the hardest bug/issue you personally fixed?
    - Specific numbers: "reduced report time by 70%" — HOW? Be ready to explain the mechanism, not just repeat the number.
- **Evening (1 hr): FastAPI internals**
    - Pydantic models & validation, dependency injection (`Depends`), async def vs def endpoints, middleware, background tasks, Swagger/OpenAPI generation (you have this — just refresh)

Day 2 (15th July) — Gen AI / LLM Foundations (the new material)

Goal: Understand concepts well enough to hold a real conversation, not memorize buzzwords.

- **Morning (2 hrs): What is an LLM, practically**
    - Tokens, embeddings, context window, temperature, top-p, prompt vs completion
    - Difference between GPT, Claude, Gemini, Llama, Mistral at a high level (open vs closed source, API-based vs self-hosted)
    - What "fine-tuning" actually means vs prompting vs RAG (know when to use which — this is a favorite interview question)
- **Afternoon (2–3 hrs): RAG (Retrieval-Augmented Generation) — THIS IS THE #1 TOPIC THEY WILL ASK**
    - Why RAG exists (hallucination, knowledge cutoff, private data)
    - Pipeline: document → chunking → embedding → vector store → retrieval → prompt augmentation → LLM response
    - Chunking strategies (fixed size, semantic, overlap)
    - Embedding models (OpenAI embeddings, sentence-transformers)
    - Vector databases: **Pinecone, Chroma, FAISS, Weaviate, pgvector** — know what they do and pick ONE to say you've used
    - Similarity search: cosine similarity vs dot product (just conceptually)
- **Evening (1–2 hrs): LangChain / LlamaIndex basics**
    - What LangChain is: chains, agents, tools, memory
    - What "AI Agents" means: an LLM that can decide to call tools/functions in a loop

- Prompt Engineering: zero-shot, few-shot, chain-of-thought, system prompts

Day 3 (16th July) — Hands-On Build + Cloud AI Services

Goal: Build ONE small working RAG project so you can say "I built this" with a straight face.

- **Morning–Afternoon (4–5 hrs): Build a mini RAG app**
  - Use Python + FastAPI (your strength!) + LangChain + a free/open embedding model + FAISS or Chroma (local, free vector DB)
  - Take a PDF (e.g., your own resume or any doc) → chunk it → embed it → store in FAISS → build a `/ask` FastAPI endpoint that retrieves relevant chunks and calls an LLM API (use a free-tier key: OpenAI, Gemini, or Claude API) to answer questions about the doc
  - This single project lets you answer confidently: "I built a document Q&A system using FastAPI, LangChain, and FAISS with OpenAI embeddings" — a genuine, defensible hands-on claim
- **Evening (1–2 hrs): Cloud AI services — just terminology level**
  - Azure OpenAI Service (what it is: Azure-hosted GPT models)
  - Azure AI Services (Cognitive Services — vision, speech, language APIs)
  - AWS Bedrock (AWS's managed LLM service) — mention briefly, you know GCP/AWS S3 already
  - Google Cloud AI (Vertex AI) — brief

Day 4 (17th July) — MLOps Basics + Mock Interview + Rest

Goal: Cover the last gap, then consolidate — don't cram new things the night before.

- **Morning (2 hrs): MLOps — connect it to what you already know**
  - Docker (if you haven't used it — learn just enough: build an image, run a container, Dockerfile basics for a Python app)
  - CI/CD — you already know this conceptually from Agile/Jira experience; just map it to model deployment (deploy model as an API, version it, monitor drift)
  - MLflow — just know: it tracks experiments, model versions, and metrics
  - Kubernetes — conceptual only: orchestrates containers at scale; you won't be expected to be an expert given your profile
- **Afternoon (2 hrs): Coding practice** (see Section 3 below)
- **Evening: Mock interview with yourself / a friend**
  - Say your 2-minute self-intro out loud 3 times
  - Practice explaining RAG on a whiteboard/paper in under 90 seconds

- Sleep early. Don't learn anything new after 8 PM.

---

2. What They'll Likely Ask a 3-YOE Candidate With Your Background

Given your resume, expect the interview to be **~60% your actual backend/data experience, ~40% Gen AI fundamentals** — they're checking if you can *learn and apply* Gen AI on a strong Python/backend foundation, not testing you like a senior ML researcher.

Almost certain:

- Walk me through your FastAPI project (Stelvio) end to end
- Walk me through your Airflow/Snowflake pipeline (AstraZeneca)
- What is RAG? Why would you use it over fine-tuning?
- What's the difference between fine-tuning and prompt engineering?
- Explain the RAG architecture pipeline step by step
- What vector databases do you know?
- Write a Python function to [some string/list/dict manipulation]
- Design a REST API for [some scenario] — they'll want to see Pydantic models, endpoint design
- What is prompt engineering? Give an example of few-shot prompting

Likely:

- Explain embeddings in simple terms
- What's the difference between GPT, Claude, and open-source models like Llama/Mistral?
- How would you build a chatbot using an LLM + your company's internal documents?
- Have you used LangChain/LlamaIndex? Explain a chain or agent
- SQL query question (given your Snowflake/dbt background)
- How do you handle API rate limits / errors when calling an LLM API?

Possible (be aware, don't over-invest):

- What is Docker? Have you containerized anything?
- What is MLflow / model monitoring?
- Kubernetes basics

---

3. Coding Question Types to Practice

Given your profile (Python backend + data), expect **easy-medium** difficulty, not competitive-programming-hard. Focus on:

1. **String/List manipulation** — reverse a string, check palindrome, find duplicates, anagram check, word frequency count
2. **Dictionary/Hashmap problems** — two-sum, group items by key, count occurrences
3. **Pandas/data problems** — given a CSV/dataframe, group by, filter, aggregate (very likely given your ETL background)
4. **API/REST design question** — "design an endpoint to upload a document and query it" (this connects directly to RAG — practice this one specifically)
5. **OOP design** — design a class structure (e.g., model a Booking system, reflecting your Stelvio project)
6. **SQL** — joins, group by, window functions (given Snowflake/dbt experience)
7. **Basic RAG pseudocode** — they may ask you to write pseudocode/actual code for: load doc → chunk → embed → store → retrieve → generate. **Practice writing this from memory.**

Practice platform suggestion: LeetCode Easy tag "Python" + "Pandas" problems, 1 hour/day is plenty — don't grind Hard problems, they're not relevant for this role level.

---

4. How to Bridge the Gap Honestly (Important)

When asked "what's your Gen AI experience?" — don't oversell, don't undersell. Use this structure:

> "My core professional experience is in Python backend development and data engineering — building FastAPI/Django APIs and Snowflake/Airflow pipelines at Accenture. I haven't yet worked on Gen AI in a production role, but I've been actively building hands-on skills — for example, I built a RAG-based document Q&A system using FastAPI, LangChain, and FAISS. My backend and data pipeline experience translates directly: RAG pipelines are essentially data pipelines feeding an LLM, and I already have strong fundamentals in API design, data validation, and pipeline orchestration that Gen AI application development builds on."

This is honest, confident, and turns your "gap" into a narrative of relevant transferable skill + initiative.

---

5. Quick Reference Sheet (Skim Morning of 18th July)

- **RAG pipeline:** Doc → Chunk → Embed → Vector Store → Retrieve top-k → Augment

prompt → LLM → Response

- **Fine-tuning vs RAG vs Prompt Engineering:** Fine-tuning = retrain weights on custom data (expensive, static knowledge). RAG = keep base model, fetch relevant external data at query time (cheap, dynamic knowledge). Prompt engineering = just craft better inputs, no retraining/retrieval.
- **Vector DBs:** FAISS (local/free, Meta), Chroma (local, easy), Pinecone (managed cloud), pgvector (Postgres extension), Weaviate
- **LangChain building blocks:** LLM wrapper, PromptTemplate, Chains (sequence of calls), Agents (LLM decides which tool to call), Memory (conversation history)
- **Agents:** an LLM in a loop that can call tools/functions and decide next steps based on results
- **Common cloud AI services:** Azure OpenAI (GPT models on Azure), AWS Bedrock (managed LLMs on AWS), Vertex AI (Google's ML/AI platform)

---

6. Logistics Reminder for 18th July

- Carry multiple physical copies of your resume
- Carry ID proof + any offer/experience letters from Accenture
- Dress formally — it's an in-person walk-in
- Reach TCS Magnum Chennai with buffer time
- Carry a notebook/pen — you may be asked to write pseudocode on paper

Good luck — your Python/backend foundation is genuinely solid. Lead with that confidence, and be honest and enthusiastic about the Gen AI learning curve.